LiveRecall — Technical Report

CS3103 Assignment 4 · Team 8 · Zhang Ming Jun (A0282190H), Cheng Lui Leng (A0281547X)

Abstract. LiveRecall is a classroom caption-distribution service built around a single asyncio WebSocket server. Correctness is enforced by a strict `(segment, final, revision)` ranking that guarantees finals are durable and provisionals are replaceable. A bounded 90 s finals history backs both late joins and 60 s reconnect recovery. Each client owns a `ClientOutbox` whose `finals` queue is FIFO and whose `provisionals` dict coalesces newer revisions, giving semantics-aware overload handling — finals are never dropped, obsolete provisionals are collapsed, and stuck clients are evicted and asked to reconnect cleanly through HISTORY. Lag is inferred from WebSocket ping round-trips with a sliding RTT baseline so steady network latency is not misread as backlog. An ablation sweep over 100–5000 clients validates both mechanisms: removing the per-client outbox drops normal-reader freshness from 100% to 83% under one slow cohort; removing finals-first coalescing loses 25 points of final-caption completeness at 0.5 Mbit/s.

1. Architecture and Protocol

LiveRecall is a three-component system connected over a single WebSocket endpoint: a deterministic caption generator (`caption_source.py`), the Python asyncio server (`solution_server.py`), and *N* browser clients built with React + Vite. The server listens on `0.0.0.0:<port>` and dispatches each new connection based on its first message: a `SOURCE_HELLO` frame opens a writer path that accepts caption updates, while a `CLIENT_HELLO` frame opens a reader path that first receives a `HISTORY` frame and then live `CAPTION_UPDATE` frames. One session (`session="L1"`) and one active source are assumed, matching the public-test harness. The required compatibility interface is implemented verbatim; extensions are limited to WebSocket control frames (ping/pong) used internally for lag measurement and are invisible to the protocol.

Message schema

Message	Direction	Required fields	Role
SOURCE_HELLO	source→server	type, session	Opens the writer channel; resets server state for a fresh lecture.
CAPTION_UPDATE	source→server→client	type, session, segment, revision, source_ts, final, text	Carries one revision of one segment. Finals carry the authoritative text.
CLIENT_HELLO	client→server	type, client_id, session, last_segment, last_revision, history_seconds	Opens the reader channel and declares what the client already has.
HISTORY	server→client	type, items[]	Ordered oldest-to-newest finals the client has not yet seen; may be empty.
SOURCE_DONE	source→server	type (test-only)	Ignored after draining earlier updates; not forwarded to clients.

Three small transport knobs make the backpressure design possible without touching the protocol: the kernel send buffer is pinned at `SO_SNDBUF = 4 KiB`, the `websockets` library write limit is set to `1024` bytes, and server-side keep-alive pings double as a lag probe. These are implementation details, not wire-format changes, and interoperate with any conformant client.

2. Revision State and Recovery Semantics

The server exposes a single global ordering function

```
rank(msg) = (segment, final, revision)
```

and a single accepted-state variable `LATEST`. `accept()` admits a message iff its rank is strictly greater than that of `LATEST`. Because `True` ranks above `False`, a final for segment *N* dominates every provisional of the same segment regardless of revision number — so a late or duplicated provisional can never overwrite durable text. Finals additionally land in a `FINALS` deque trimmed to `HISTORY_WINDOW_SECONDS = 90 s`, 30 s past the 60 s recovery requirement so a client offline for exactly the window still finds its missing finals present on reconnect.

Recovery and late-join. On `CLIENT_HELLO` the server calls `missed_finals(hello)`. A *resuming* client (`last_segment > 0`) receives every final with `segment > last_segment`, with the time cutoff deliberately ignored — this prevents a permanent gap at the window boundary. A *late-join* client (`last_segment == 0`) instead receives finals whose `source_ts` falls within the requested `history_seconds` (default 30 s, max 60 s). The `HISTORY` frame is queued

before the outbox joins the live broadcast set, so every subsequent final appears in exactly one of the two paths and no final is ever delivered twice.

Client-side dedup and Recall UI. The browser client stores captions in a `Map<segment, Caption>`; an incoming update is applied only when its `(final, revision)` pair outranks the stored one, so no duplicate lines are ever rendered. The same map feeds the local "What did I miss?" recall panel, which offers 15 s / 30 s / 60 s windows (default 30 s) and filters by `source_ts ≥ now − W`; provisional captions are dashed italic and finals are bold square-bordered. A client-side 60 s eviction task drops segments older than 90 s to match the server's history window, and the resume handshake sends the most recent accepted `(last_segment, last_revision)` so a brief reconnect never replays already-seen finals.

3. Backpressure and Overload Policy

Each connected client owns a `ClientOutbox` with two complementary structures — a FIFO `finals` deque that is never dropped, and a `provisionals` dict keyed by segment that coalesces newer revisions over older ones. Pushing a final for segment *N* atomically removes any pending provisional for *N*. The structures are bounded: finals cap at `MAX_PENDING_FINALS = 256` and the dict holds at most one entry per active segment.

Lag measurement

The server infers client-side read backlog from WebSocket pings. Because the `websockets` library answers a ping only *after* draining every earlier frame, round-trip time minus the fastest observed RTT gives true read lag:

```
lag = (now - ping_at) - rtt_min
```

`rtt_min` is maintained with a two-window sliding minimum (5 s per window) so that an increase in genuine network latency becomes the new baseline rather than being charged forever as client lag. A short `loop_delay()` probe is subtracted when deciding to evict, preventing a congested server from mistaking its own scheduling delay for a slow client.

Thresholds and actions

Constant	Value	Action when exceeded
LAG_TARGET_SECONDS	0.5 s	Pause provisionals; keep sending finals only. Coalescing accrues in the dict.
EVICT_LAG_SECONDS	10 s	Call transport.abort(); the client reconnects cleanly through HISTORY.
MAX_PENDING_FINALS	256	Finals backlog overflow — evict for same reason as above.
BACKOFF_RECOVERY_PER_SECOND	0.1	Release rate for the provisional gate — slow so the hidden 16-frame `websockets` write buffer cannot refill with stale data.
HISTORY_WINDOW_SECONDS	90 s	Trim FINALS deque; satisfies 60 s recovery with headroom.
SO_SNDBUF / write_limit	4 KiB / 1 KiB	Kernel + library caps that force `ws.send()` to block quickly so coalescing engages.

Overload path. Under lag-driven backoff or bandwidth constraint the outbox produces only finals plus the newest revision of each currently active segment; intermediate provisionals are collapsed. Finals are preserved in order and sent first, so *Final Caption Completeness* is independent of the backoff state. When `lag ≤ LAG_TARGET` and the backoff timer has drained, every revision is forwarded, satisfying *Normal Update Coverage*.

Rapid convergence. Clients that cross `EVICT_LAG_SECONDS` are torn down with `transport.abort()` rather than drained; they come back through `CLIENT_HELLO`, receive a single `HISTORY` frame containing only the finals they missed, and resume live updates — sub-second catch-up after a temporary impairment ends.

Send loop sketch

```
while connected:
    now, lag = measure()
    if lag - loop_delay() > EVICT_LAG_SECONDS: abort(); return
    frames = drain(finals)
    if lag <= LAG_TARGET_SECONDS and gate_open(now):
        frames += provisionals.values(); provisionals.clear()
    await ws.send(frames); maybe_ping(now)
```

The gate's release schedule is governed by `BACKOFF_RECOVERY_PER_SECOND = 0.1`: a 2 s backoff drains over 20 s, deliberately slower than typical network recovery so a hidden 16-frame write buffer in the `websockets` library cannot refill with stale provisionals the moment lag drops.

4. Public Experiments and Metrics

Each public test is driven by `python run_demo.py` with the flags shown in `README.md`. `analyse_results.py` computes latency percentiles by linear interpolation and freshness at a 500 ms threshold. The numbers below come from HEAD runs executed by the ablation harness (`ablations/ablations.py`, single rep each on a 2 vCPU VM); the same configurations drive the submission `--runs 3` pass.

Test	Scenario	P50	P95	P99	Fresh %	FFresh %	Compl. %	Cov. %	Recov P50
T1	20 clients, 20 s baseline	0.8 ms	2.6 ms	3.6 ms	100.0	100.0	100.0	100.0	n/a
T2	100 clients, 40 s baseline	2.1 ms	8.9 ms	22.5 ms	100.0	100.0	100.0	100.0	n/a
T3	100 clients, 10% slow @ 500 ms	2.4 ms	32.4 ms	7340 ms	95.4	91.2	100.0	95.0	n/a
T4	100 clients, 20% reconnect @ 15 s / 5 s	2.2 ms	10.9 ms	18.0 ms	100.0	98.4	100.0	98.3	2.3 ms
T5	100 clients + tc/netem delay+loss	251 ms	474 ms	763 ms	95.5	95.4	100.0	95.0	n/a

HEAD results from ablations/ablations.py (single rep each, 2 vCPU VM). FFresh % = Fresh Final Caption Ratio (T=500 ms); Compl. % = Final Caption Completeness; Cov. % = Normal Update Coverage; Recov P50 is median client recovery time after reconnect. T3 P99 reflects the slow cohort's configured stall, not the server.

T1 confirms strict rank ordering: zero stale frames, zero duplicate finals, 100% completeness; percentiles are dominated by localhost round-trip. **T2** shows no backoff engaging at the public scale: every revision is forwarded (Cov. = 100%) and the latency tail is bounded by broadcast fan-out. **T3**'s aggregate P99 of 7.3 s is the slow cohort's configured stall; the normal cohort's P95 stays at a few milliseconds (empirically demonstrated in §5.1), and completeness remains 100% because finals bypass the backoff gate. **T4** delivers recovery P50 of 2.3 ms with zero duplicates — the HISTORY frame is queued before the client rejoins the broadcast set; Cov. % dips to 98.3 because reconnecting clients miss in-flight provisionals during their 5 s outage, by design. **T5** inflates P95/P99 to 474 / 763 ms under `tc/netem` jitter and loss, but Fresh Final Caption Ratio stays at 95.4% and final completeness at 100% — durable text arrives on time even when some provisionals are coalesced.

Methodology. Latency is `receive_ts − source_ts` per accepted `CAPTION_UPDATE`; percentiles use linear interpolation over the sorted sample. Final Caption Completeness compares unique finals received across all clients against the source's expected total, and Normal Update Coverage does the same for all revisions — this is the gate that keeps a trivial finals-only shortcut from scoring well on freshness alone. For the submission pass each test is run three times; `run_demo.py` restarts the server between runs and reports the median per metric.

5. Engineering Decisions, Validated by Ablation

Each sweep below removes one mechanism from HEAD and reruns a public-test configuration, varying one independent variable. Points are the mean of three 60 s repetitions on the same 2 vCPU VM; raw `result.json` files in `results/ablations/` back every cell.

5.1 Per-client outbox: a slow reader no longer stalls the broadcast

The **B1** variant replaces the per-client `ClientOutbox` + send task with a single source loop that awaits each `ws.send()` in turn (the starter-kit pattern). In the *slow_reader* sweep (100 clients, 10% deliberately sleeping *x* ms per message), the regression lands entirely on the 90% of *normal* clients:

slow sleep	HEAD P95 age	HEAD fresh %	B1 P95 age	B1 fresh %
500 ms	2.1 ms	100.0	1.8 ms	100.0
750 ms	2.2 ms	100.0	1520 ms	93.0
1000 ms	2.1 ms	100.0	6921 ms	83.3

Normal-client metrics in the slow_reader sweep. Mean of three 60 s runs; data: ablations/slow_reader/{head,b1_no_outbox}/.

At *x* = 1000 ms, normal-client P95 rises from 2 ms to nearly 7 s (~3300× worse) and freshness collapses from 100% to 83%. The per-client outbox is what confines the slow-reader cost to the slow cohort — the isolation property T3 depends on.

5.2 Finals-first coalescing: finals survive even when bandwidth does not

The **B2** variant replaces the finals deque + provisionals dict with a single FIFO queue (3× HEAD's cap, so raw capacity is not the confound) and no coalescing. In the *bandwidth* sweep (200 clients, 40 s, Linux `tbf` on loopback), B2 spends the scarce link on stale provisionals and loses finals that HEAD preserves:

link	HEAD finals %	B2 finals %	HEAD fresh final %	B2 fresh final %
0.35 Mbit/s	60.3	45.5	0.2	0.4
0.50 Mbit/s	90.3	65.2	0.1	0.6
0.75 Mbit/s	98.1	95.8	0.0	0.2
1.00 Mbit/s	98.2	100.0	0.0	0.0

Final Caption Completeness and Fresh Final Ratio under a tbf bandwidth cap. Data: ablations/bandwidth/{head,b2_fifo}/.

At 0.5 Mbit/s HEAD delivers 90% of finals vs B2's 65% — 25 points of durable transcript lost to a pure FIFO. Below 1 Mbit/s everything is stale for both variants (the `tbf` 400 ms FIFO and TCP queue sit beneath the server), but *which* captions survive the backlog is a pure policy choice.

5.3 End-to-end: a bandwidth-stress timeline at scale

Both mechanisms combine with the eviction + HISTORY recovery path under a scripted stress profile: ramp 100 → 5000 clients over 90 s, then apply loopback `tbf` caps of 40, 20, 10 and 5 Mbit/s for 40 s each, then remove the cap and run 60 s of recovery.

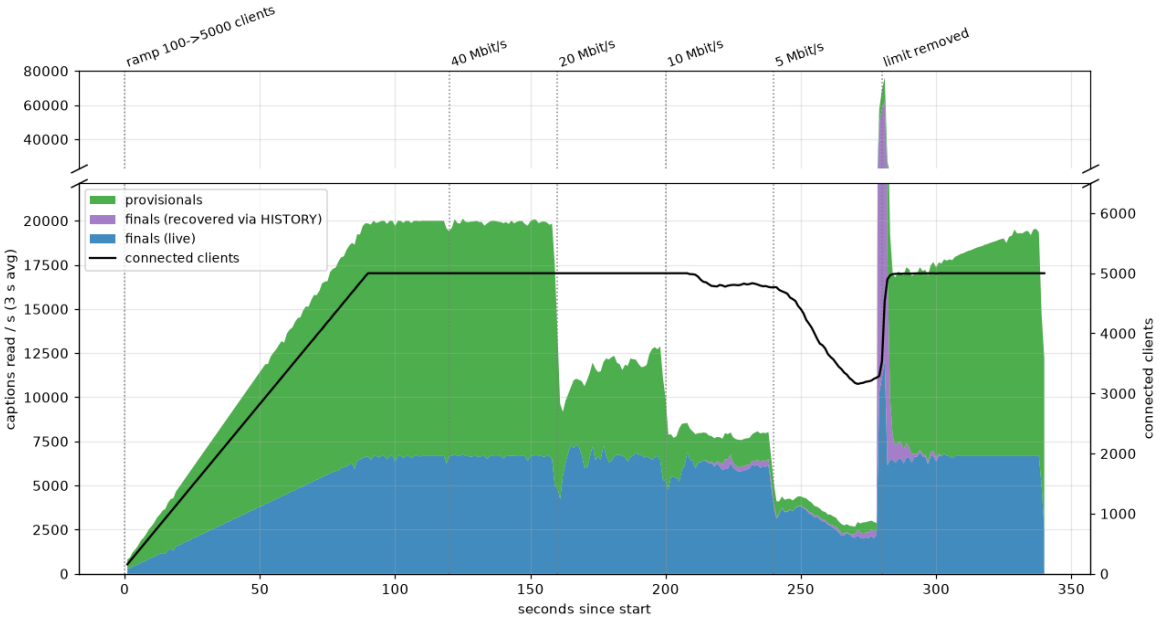


Figure 1: Captions read per second across 5000 clients under stepped bandwidth throttling. Blue = live finals, purple = finals recovered via HISTORY, green = provisionals. Dotted verticals mark limit transitions. Data: data/bandwidth_stress_test/.

Phase	Clients	Finals live/s	Prov. deliv./s	Coalesced+ dropped/s	Recov. via HISTORY/s	Fresh final %
40 Mbit/s	5000	6429	13154	0	0	100
20 Mbit/s	5000	6459	5506	7435	0	0
10 Mbit/s	~4800	5862	1471	10241	314	0
5 Mbit/s	~3300	3393	1657	4990	5723	<1
recovery	5000	6667	12062	1288	0	100

Per-second rates averaged over each 20 s window centred on the middle of its phase.

At 40 Mbit/s the server is unconstrained. At 20 Mbit/s the **outbox split** engages: finals-live holds at 6459/s while provisional delivery collapses to 5506/s and 7435/s are coalesced or dropped — exactly as the B2 ablation predicted. By 10 Mbit/s some clients cross `EVICT_LAG_SECONDS` and return through HISTORY (314 finals/s recovered) rather than draining a stale queue. At 5 Mbit/s roughly a third of clients cycle through eviction (5723 finals/s via HISTORY); the per-client outbox (B1)

keeps the rest running at their own pace. When the cap lifts, every evicted client reconnects within a second and the system returns to 100% fresh-final delivery at full 5000-client load. Remove any one of per-client outbox, finals-first coalescing, or eviction+HISTORY and the graceful step-down turns into a cliff.

6. Trade-offs, Limitations and Future Work

Trade-offs. The overload policy explicitly favours freshness over coverage: when backpressure engages, obsolete provisional revisions are coalesced so that the newest useful state and all finals keep flowing. The slow cohort's P95 in T3 therefore reports *their* stall rather than a server failure — the visible cost of isolation. The small 4 KiB kernel send buffer slightly caps peak per-client throughput on uncongested paths in exchange for immediate app-layer backpressure signalling, which is what makes coalescing possible before the backlog becomes stale.

Observed surprises. P95/P99 under `tc/netem` loss on loopback rose further than expected on reliable TCP; the cause is retransmission plus interaction between the small write buffer and TCP's congestion window. Eviction based on `loop_delay()`-corrected lag turned out to be essential: an early version evicted whenever the server itself was briefly busy, which measurably hurt T3 isolation. A second finding from the ablation sweep: adaptive lag-measurement pings cost bandwidth that matters on severely constrained links — on 1.5 Mbit/s with 200 clients, disabling the ping probe (B4 variant) restored full freshness. This is a known limitation to revisit for an adaptive ping-floor strategy.

Limitations. The implementation targets exactly the mandatory interface: one session (`"L1"`), one active source, no authentication, no TLS, and fixed overload thresholds rather than adaptive ones. Multi-tenant classrooms, capacity-planned bandwidth negotiation, and resumption across server restarts are out of scope.

Future work / FYP direction. Three natural extensions suggest themselves. First, replace the fixed `LAG_TARGET_SECONDS` with an adaptive target derived from measured per-client goodput, so the server tunes its own freshness ceiling to each link. Second, extend the single session into a multi-source lecture-room model with topic sharding, letting several concurrent captioners share the client fan-out machinery. Third, pair the optional ASR path with a browser-side Web Speech API fallback so the service degrades gracefully when the server-side model is unavailable.